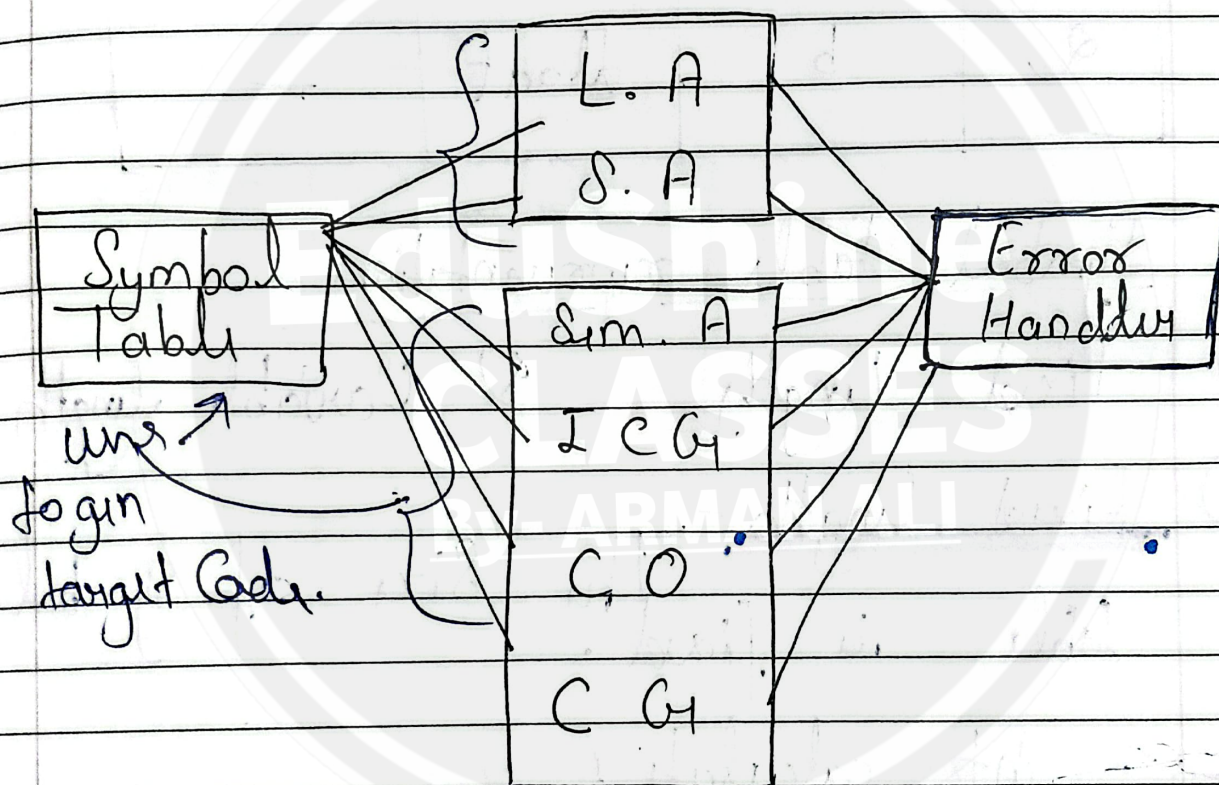


UNIT-4

* Symbol Table *

* Symbol table is a data structure and maintained by compiler in order to store information about variables, function, class, object etc.



Format of Symbol Table -

uns in following type information in Symbol Table -

- 1) Data type
- 2) Name
- 3) Scope
- 4) Address
- 5) Other Attribute

Eg static int a;
float b;

Symbol table -

S.No	Name	Type	Attribute
1,	a	int	Static
2,	b	float	-

* Symbol table representation -

1) Fixed Length 2) Variable length

① Fixed length - The size of Symbol table is fixed.

Ex int calculator;
int sum;
int a, b;

S.No	Name	Type
1,	calculator	int
2,	sum	int
3,	a	int
4,	b	int

} memory is wasted.

② Variable Length - The size of symbol table is variable. ✓ mood

Eg-
 int calculat;
 int sum;
 int a, b;

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
c	a	l	c	u	l	a	t	e	\$	s	u	m	\$	a	\$	b	\$

S. Index	length	type
0	10	int
10	4	int
14	2	int
16	2	int

Operation on Symbol table -

i) Insert(): Add a new entry when a variable or function is declared.

Ex $\text{insert}(\overset{\text{name}}{a}, \overset{\text{type}}{\text{int}})$

ii) lookup() - $\text{lookup}(\text{Symbol})$ -
 $\text{lookup}(a)$

3) Delete() - delete (Symbol)
delete (a)

4) Scope mgmt - local & global variable

Example -

```

int value;
void one ()
{
    int a;
    int b;
    {
        int c;
        int d;
    }
    int e;
    {
        int f;
        int g;
    }
}

void two ()
{
    int x;
    int y;
    {
        int p;
        int q;
    }
    int r;
}
  
```

global info (Symbol Table)

value	var	int
one	fn	void
two	fn	void

a	var	int
b	var	int
e	var	int

x	var	int
y	var	int
r	var	int

p	var	int
q	var	int

c	var	int
d	var	int

f	var	int
g	var	int

* Implementation of Symbol Table -

There are many ways, but here are the most common -

① Linear List / Array -
Store entries one by one in a list or array.

* Simple but slow when searching large no. of symbols.

Operation -

Insert ✓
Search ✓

Add →	Name	Attribute	Searching ↓
	id ₁	info ₁	
	id ₂	info ₂	
	id _n	info _n	

② Self Organizing list -
This symbol table implements using linked list. A link field added to each record.

	Name	Attr	link
	id ₁	info ₁	
	id ₂	info ₂	
→	id ₃	info ₃	
	id ₄	info ₄	
Avl →			

③ Binary Search Tree -

* Symbols are stored in a tree like structure.

* Ex - int a, n, y;



④ Hash table -

Hashing is most powerful in hashing are symbol table maintained - in two tables

Hash table

Symbol table

		Name	Attr	Link
0	sum	sum	int	
1	c	c	int	
2	y	y	int	
...	...	...	...	
k-1	ava	ava	float	
...	...	...	...	

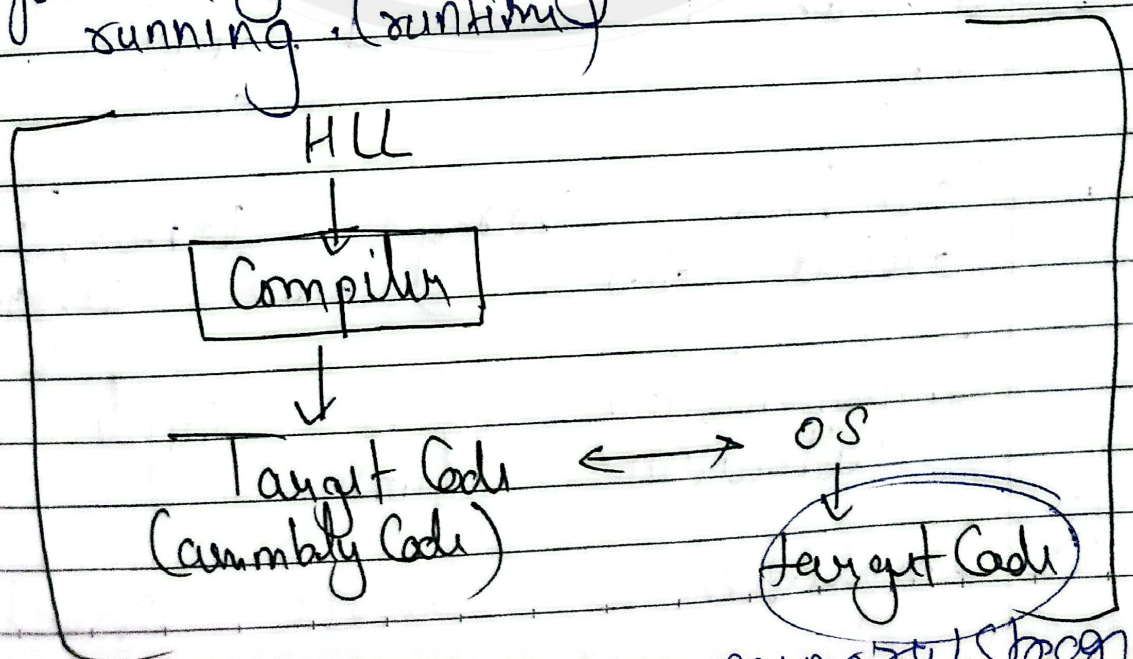
Aut →

* Run time Storage Administration -

When your program runs, it uses memory to store things like variables, functions, arrays etc.

Run time storage administration means -

→ How the Compiler manages memory while it's running (runtime)

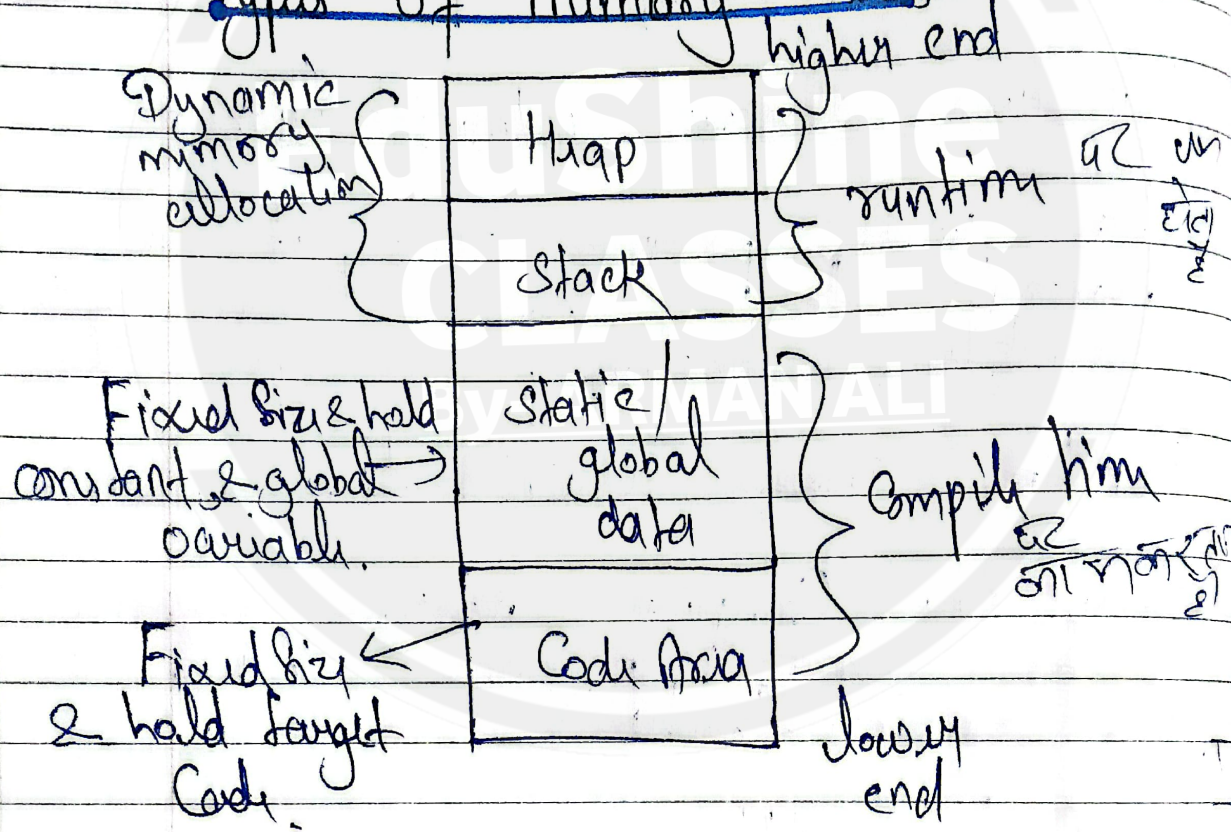


Why is it important?

Because memory is limited and the program keeps running and space as it runs, so we need a system to

- Allocate memory
- Free memory

Types of Memory Areas



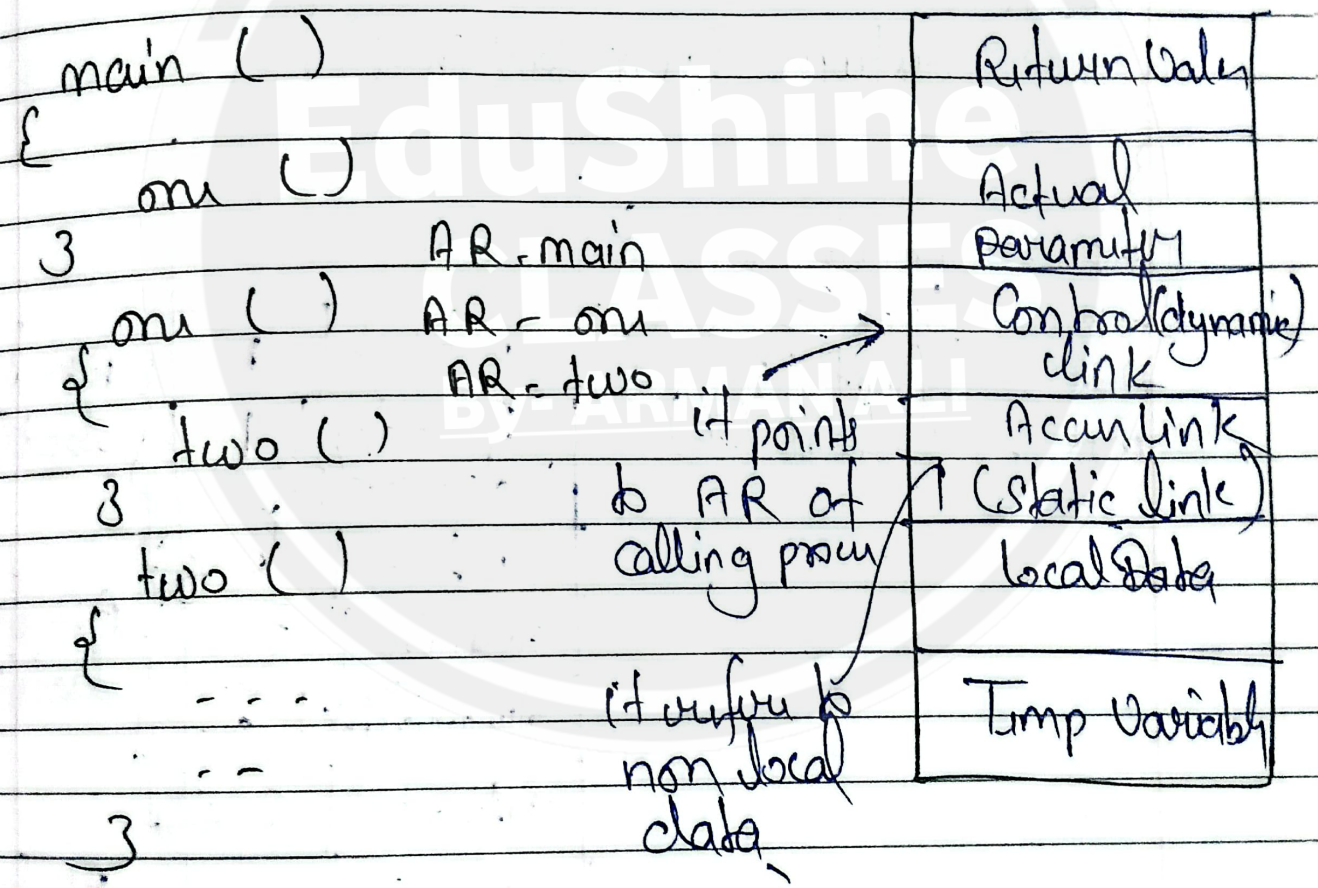
Run time storage administration divided into two categories -

- ① Implementation of a simple stack allocation scheme.
- ② Implementation of block structure.

① Simple stack allocation scheme

When your programs run & call functions, the memory for each function local variable, parameter, return address etc is stored in a stack.

This method of managing memory is called stack allocation.



Temp Variable


```

Ex: main()
{
    int f;
    f = fact(3);
}

int fact(int n)
{
    if (n == 1)
    {
        return 1;
    }
    else {
        return (n * fact(n-1));
    }
}

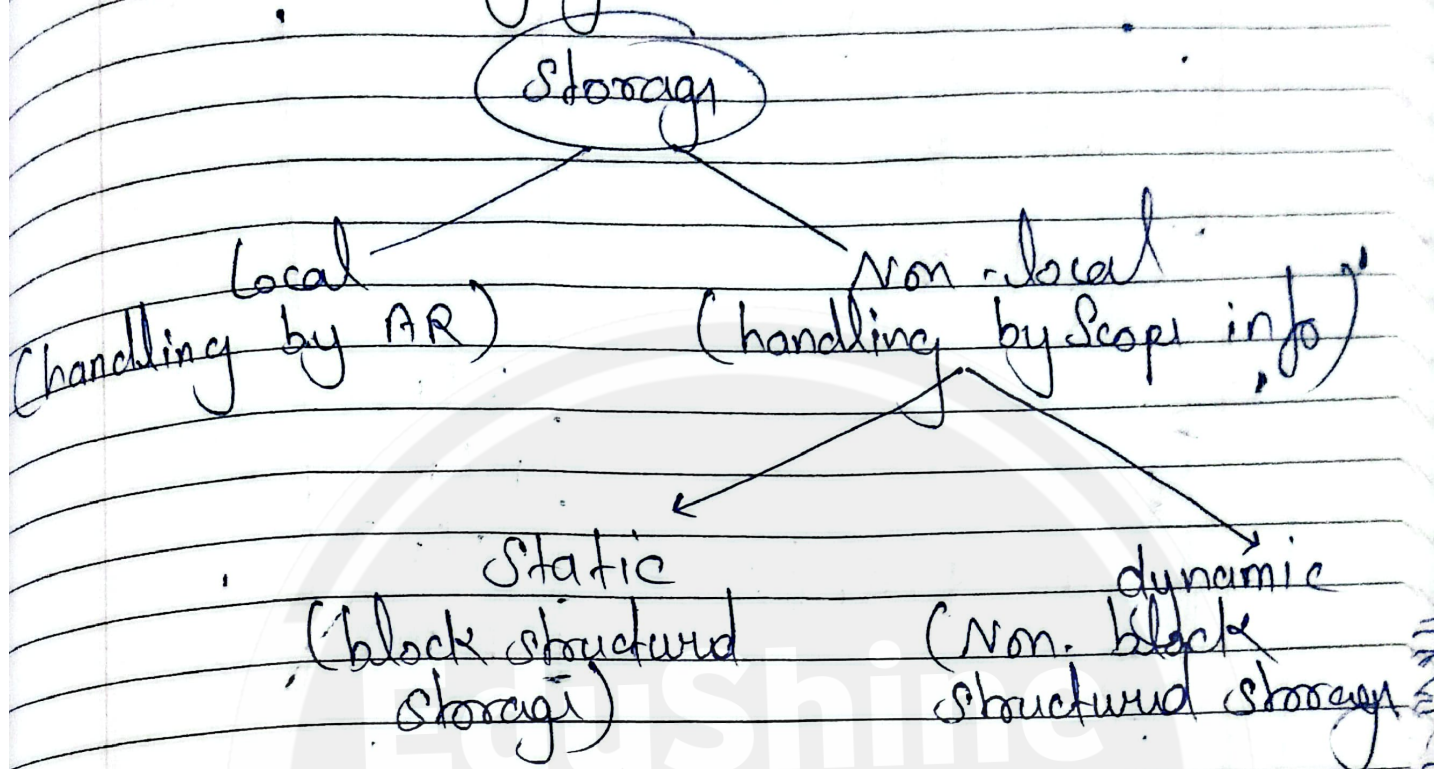
```

AR : main() }
 AR : fact(3) }
 AR : fact(2) }
 AR : fact(1) }

return val	1	AR for fact(1)
Parameter	1	
dynamic link		
return val	2	AR for fact(2)
Parameter	2	
dynamic link		
return val	6	AR for fact(3)
Parameter	3	
dynamic link		
return val	6	AR for main()
local	f	

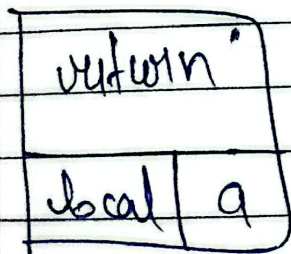
Stack

②. Implementation of Block-structured Language



Local data

```
A ( )  
{  
  int a;  
}
```

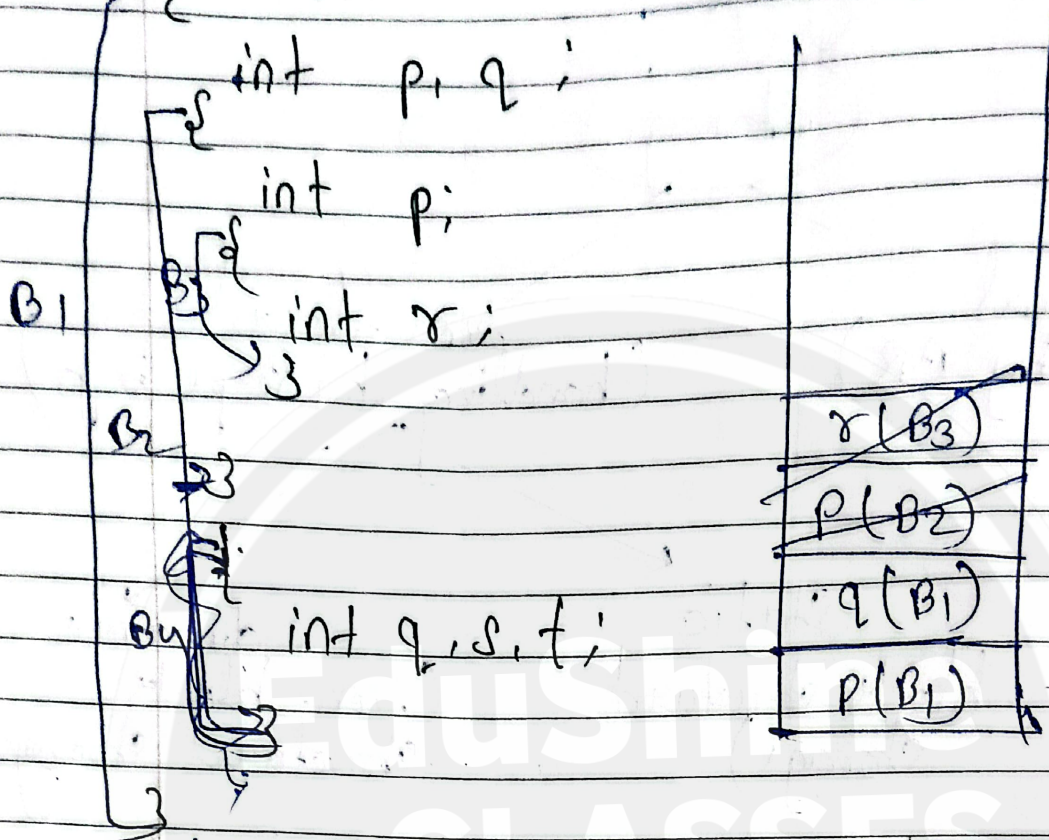


Non-local data

```
fun ( )  
{  
  {  
    int a, b;  
    int x, y;  
  }  
  {  
    int c, d;  
  }  
}
```


Static Scope Rule - (Lexical Scope)

Ex- scope_int()



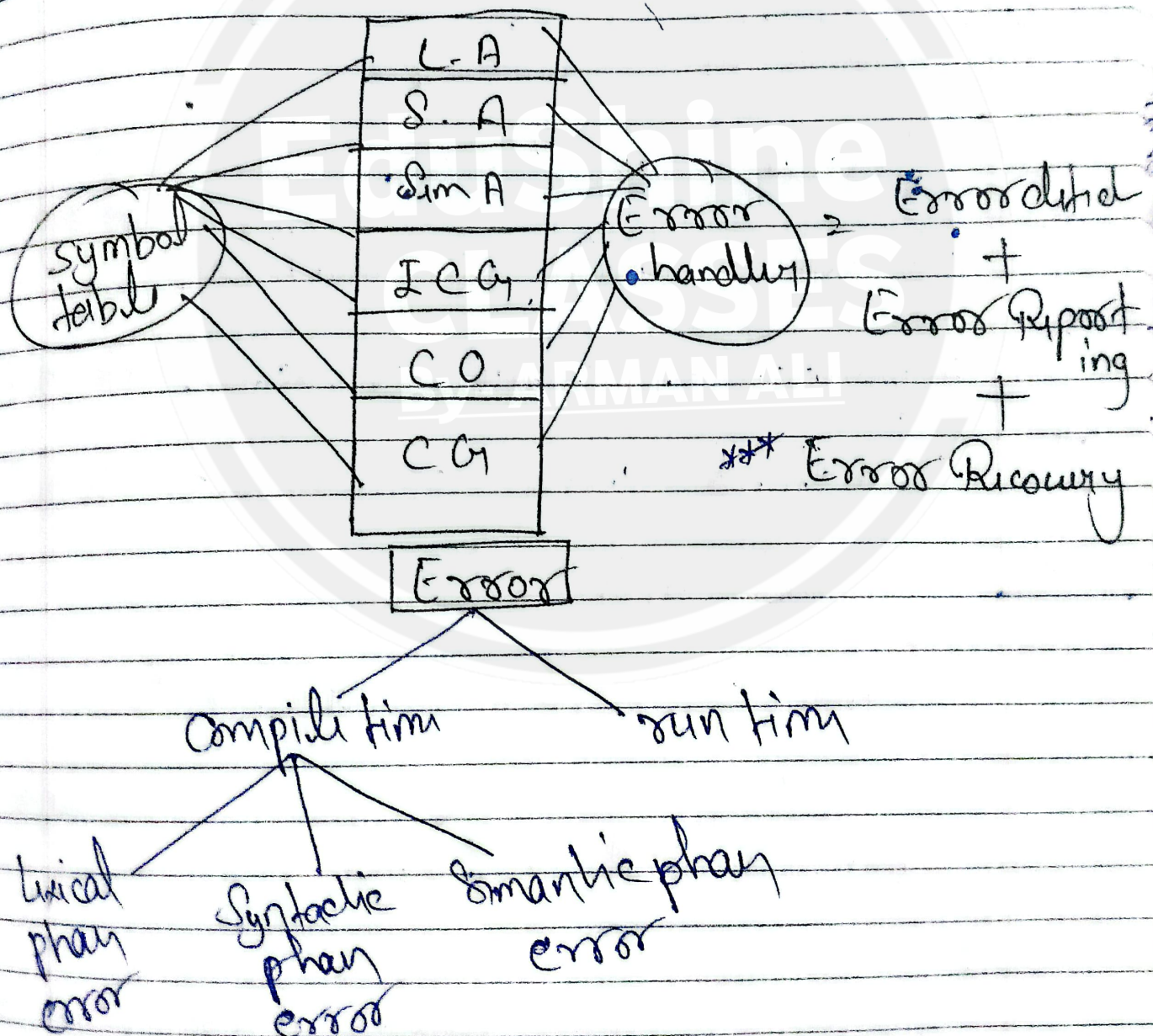
t(Su)
s(Bu)
q(Bu)
q(B1)
p(B1)

* Error Detection & Recovery

When you write code, sometimes you make mistakes, these mistakes are called errors.

The Compiler has to -

- 1. Detect the Error ✓
- 2. Handle the Error -



start with error and E

Error Recov. method	Lexical phase Error	Syntactic phase Error	Semantic phase Error
Panic mode	✓	✓	X
Phases level	X	✓	X
Error Production	X	✓	X
Global Production	X	✓	X
Using Symbol table	X	X	✓

① Lexical phase Error

- i) Exceeding length of identifier. int sum
- ii) Appearance of illegal character. → int q
- iii) Unmatched string or comment hello

Ex.

```
void main ()
{
    int a, @, $;    variable */
    a = 10;
    printf ("%d", a);
}
```

Now Recovery using Panic Mode

Panic mode method is about skipping incorrect characters until the compiler finds something valid.

Ex: i) `int a, bb, sum, xx`

ii) `int bb = 10;`
`int = 10;`

② Syntactic Phases Error:

i) Missing parenthesis - `if (n > 3`

ii) Missing Operator - `a + bc`

iii) Misspelled keyword - `switch (ch)`

iv) Colon instead of ; - `a = 1 : x a = 1; ✓`

v) Extra blank space - `sw_itch - x`

Example -

`if (n > 10`

`printf("Hello");`

Problem -) is missing - ✓

* Syntactic Error Recovery -

① Panic mode

Panic mode: Skip symbols until a synchronizing or $\{ \}$ is found

Ex if (n > 10
 printf("Hi") ;

Compiler skips till it finds ,

② Phonetic Level Recovery

• Replace or insert a few blocks to fix the problem & continue.

```
Ex. if (n > 10)
{
    printf("Hi");
}
```

3
Compiler insert } $\Rightarrow$ if $(n > 10)$ do
fix the structure

③ Error Production -

grammar production and make
an augmented grammar &
parse the i/p.

Ex- A rule like if n) can be added just to catch the mixing (and give a helpful msg.

④ Global Correction -

The Compiler tries to find the smallest set of changes (insert, delete, replace) to make the i/p correct.

→ Not Practical Just theoretical ✓

③ Semantic Phase Error -

Meaning is wrong.

Eg- Using a

→ Incompatible type of operands

int
 $a * b$
↓
float

→ undeclared variable -

→ Not matching actual argument with formal argument.

Eg- int a[10], b;

(a = b;

Now Recovery using Symbol table

Semantic error happens when the meaning of a statement is wrong.

- Program may look fine grammatically
- but it doesn't make sense logically

Ex- { int x = "hello" } X
{ x = x + 10; } X

→ Structure is correct but meaning is wrong

How are Semantic Errors Detected?

The Semantic Analysis phase of the Compiler checks -

- Type Compatibility
- variable declaration
- Function calls & arguments
- Scope of identifiers

& it uses the Symbol table to do this.

How Symbol Table helps?

① Undeclared Variable -

z = a + b; X

→ Symbol table says, a, b not found
(semantic error)

② Type mismatch -

int x = "hello";

→ Symbol table says - x is int, but assigned value is string.
(mismatch error)

③ Function Argument Error -

int add(int a, int b)

add(10); X

→ Symbol table says add() needs 2 arg.

Example of Recovery -

X = Y + 5 // Y is not declared

Recovery -

insert y in the Symbol table as
int y = 0 (dummy entry)

Show warning: Variable y undeclared
assumed int '0'.